Detecção de Bad Smells e Refatoração Segura

Disciplina: Teste de Software - PUC Minas (Prof. Cleiton Tavares)
Aluno: Rafael de Oliveira Caldeira Lopes **Matrícula:** 829981
Ferramentas: Jest, ESLint e eslint-plugin-sonarjs
Repositório: https://github.com/RafaelLopes1810/bad-smells-js-refactoring.git

Ponto de partida: dois ajustes na base

O enunciado pede para confirmar que 100% dos testes passam antes de começar. No repositório isso não acontecia, por dois motivos, e corrigi os dois com mudanças mínimas. **(1)** O projeto usa ES Modules (`"type": "module"`), mas o script `npm test` rodava só `jest`, que não entende `import` sem uma flag. Troquei apenas o script para `node --experimental-vm-modules node_modules/jest/bin/jest.js` (o Node mostra um aviso de recurso experimental, esperado). **(2)** Mesmo assim 1 dos 5 testes falhava: o HTML de itens comuns de admin saía como `<tr >` (com espaço) e o teste espera `<tr>`. Corrigi movendo o espaço para dentro da string de estilo, sem mudar o número de linhas nem a lógica:

```
- const style = item.priority ? 'style="font-weight:bold;"' : '';
- report += `<tr ${style}><td>${item.id}</td>...`;
+ const style = item.priority ? ' style="font-weight:bold;"' : '';
+ report += `<tr${style}><td>${item.id}</td>...`;
```

Depois disso, `npm test` deu 5 de 5 aprovados, e essa passou a ser a rede de segurança. Guardei uma cópia do original antes da correção para o diff poder ser conferido. Os testes originais não foram alterados.

1. Análise de smells

O código de produção é o `src/ReportGenerator.js`: uma classe com um construtor e um único método, `generateReport`, de 58 linhas. Li o arquivo antes de rodar qualquer ferramenta e destaco três bad smells do catálogo da aula.

Smell 1: Long Method (método longo)

O `generateReport` (linhas 11 a 68) monta o cabeçalho, decide o que cada perfil pode ver, marca itens prioritários, formata CSV e HTML, soma o total e monta o rodapé, tudo no mesmo corpo, com até 4 níveis de aninhamento e 8 `if`. Para a **manutenção**, qualquer mudança (um novo formato, uma regra de perfil) obriga a mexer nesse método inteiro e arrisca quebrar outra parte. Para os **testes**, a combinação perfil x formato multiplica os caminhos dentro de uma única função, o que dificulta cobrir tudo e achar a causa quando algo falha.

Smell 2: Duplicated Code (código duplicado)

```
// ramo ADMIN (linhas 36-37); as linhas 47-48 do ramo USER sao identicas
report += `${item.id},${item.name},${item.value},${user.name}\n`;
total += item.value;
```

A escrita da linha CSV e a soma do total aparecem repetidas nos ramos de admin e de usuário comum (e o mesmo acontece com o HTML). Para a **manutenção**, uma mudança no formato precisa ser feita em vários lugares, e é fácil esquecer um e deixar os perfis divergentes. Para os **testes**, cada cópia precisa ser verificada separadamente, e um bug corrigido só em um dos ramos passa despercebido.

Smell 3: Magic Numbers (números mágicos)

```
if (item.value > 1000) { // linha 30: limite de prioridade do admin
if (item.value <= 500) { // linha 45: limite de visibilidade do usuário comum
```

Os valores 1000 e 500 estão soltos nas condições, sem nome que diga o que significam. Para a **manutenção**, quem lê não sabe se são regras de negócio ou números arbitrários, e para mudar o limite é preciso caçar o literal. Para os **testes**, os casos de fronteira dependem de literais espalhados, e uma alteração pode ficar fora de sincronia com os testes.

Outros cheiros que anotei: Switch Statements (cadeias de `if/else` por formato e por papel espalhadas pelo método), **Comments** usados como "desodorante" (rótulos como "Seção do Cabeçalho" que só marcam responsabilidades que deveriam ser funções) e um indício de **Divergent Change** (o mesmo método muda por motivos diferentes: formato de saída e regra de acesso). Não considerei Large Class, Feature Envy nem Long Parameter List, pois a classe é pequena e nada disso se confirmou no código.

2. Relatório da ferramenta (ESLint + SonarJS)

O ESLint 8.57.1 e o eslint-plugin-sonarjs 0.21.0 já estavam nas dependências do projeto, e o `.eslintrc.json` também já veio no repositório. Ele usa o limite de complexidade cognitiva **5** (o enunciado fala em 15) e a regra extra `no-duplicated-branches`. Mantive o arquivo como veio e conferi que, com o limite 15, o resultado é exatamente o mesmo. Rodei `npx eslint src/` antes de criar o arquivo refatorado, e o resultado foi de **2 problemas (2 erros)**, ambos no original:

Linha	Regra	Mensagem
11	`sonarjs/cognitive-complexity`	Refactor this function to reduce its Cognitive Complexity from **27** to the 5 allowed
43	`sonarjs/no-collapsible-if`	Merge this if statement with the nested one

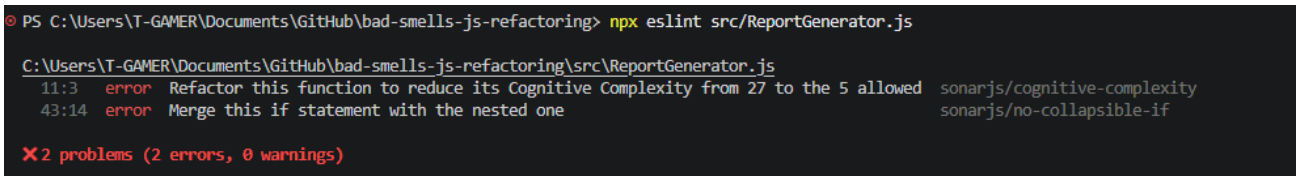


Figura 1 - Resultado do ESLint no código original.

O que o SonarJS trouxe que a análise manual poderia ter perdido: um **número** para a dificuldade de entender o método. Eu percebi que o `generateReport` era longo, aninhado e cheio de `if`, mas só a ferramenta mediu a complexidade cognitiva em **27**, mais de cinco vezes o limite, o que mostra que o problema é grave e dá uma meta objetiva para a refatoração. Ela também achou o `no-collapsible-if` na linha 43: o bloco `else if (user.role === 'USER')` só contém um `if`, que poderia ser unido com `&&`, um detalhe fácil de passar batido na leitura. Rodando medições auxiliares (limites zerados só pela linha de comando), também obtive a complexidade ciclomática do método (**15**) e os números mágicos 1000 e 500, que a configuração normal não reporta.

Por outro lado, a ferramenta **não** acusou o código duplicado (as regras de duplicação só pegam funções ou ramos idênticos, e aqui os trechos são parecidos, não iguais), nem os comentários, nem as cadeias de condicionais como smell. Ou seja, a ferramenta e a análise manual se completam.

3. Processo de refatoração

Criei `src/ReportGenerator.refactored.js` como cópia do original e `tests/ReportGenerator.refactored.test.js` como cópia do teste, mudando apenas o import de topo. (No repositório a pasta de testes se chama `tests/`, e não `__tests__/`.) O smell mais crítico foi o **Long Method** com complexidade 27, então foi nele que me concentrei. Trecho do `generateReport` original (linhas 27 a 55, resumido; as reticências marcam partes omitidas):

Antes (src/ReportGenerator.js)

```
for (const item of items) {
 if (user.role === 'ADMIN') {
 if (item.value > 1000) {
 item.priority = true;
 }
 if (reportType === 'CSV') {
 report += `${item.id},${item.name},${item.value},${user.name}\n`;
 total += item.value;
 } else if (reportType === 'HTML') {
 // ... formata a linha HTML do admin e soma o item
 }
 } else if (user.role === 'USER') {
 if (item.value <= 500) {
 // ... volta a selecionar CSV/HTML e soma o item
 }
 }
}
```

Depois (src/ReportGenerator.refactored.js)

```
for (const item of items) {
 const formatForRole = ITEM_FORMATTERS_BY_ROLE.get(user.role);
 if (formatForRole === undefined) {
 continue;
 }

 const formattedItem = formatForRole(reportType, user, item);
 if (formattedItem === null) {
 continue;
 }

 report += formattedItem;
 total += item.value;
}
```

```
function formatAdminItem(reportType, user, item) {
 if (item.value > ADMIN_PRIORITY_THRESHOLD) {
 item.priority = true;
 }
 return formatReportItem(reportType, item, user, true);
}

function formatStandardUserItem(reportType, user, item) {
 if (item.value > STANDARD_USER_VALUE_LIMIT) {
 return null;
 }
 return formatReportItem(reportType, item, user, false);
}
```

Técnicas que apliquei: Extract Method para tirar do método principal o cabeçalho, o rodapé e a formatação de cada linha (`createHeader`, `createFooter`, `formatCsvItem`, `formatHtmlItem`, `formatReportItem`), o que também eliminou a duplicação (Duplicated Code): a escrita da linha e a soma agora existem em um lugar só. **Decompose Conditional com guard clauses** (`continue` logo para itens que não se aplicam), reduzindo o aninhamento. Troquei a cadeia de `if/else` por papel por um **mapa de estratégias** (`Map`), uma forma simples de Replace Conditional with Polymorphism: cada perfil tem sua função, e um novo perfil vira uma nova entrada. Usei `Map` e não um objeto comum para que papéis desconhecidos, ou nomes como `toString`, continuem se comportando igual ao original. Por fim, **Replace Magic Number with Symbolic Constant** para 500, 1000, papéis e formatos (`ADMIN_PRIORITY_THRESHOLD`, `STANDARD_USER_VALUE_LIMIT` etc.). A classe mantém `export class ReportGenerator` e a assinatura `generateReport(reportType, user, items)`.

Fiz a refatoração em 5 passos pequenos, rodando `npm test` depois de cada um, e **todos os passos terminaram com 10 de 10 testes passando**, sem precisar desfazer nada:

Passo	Smell	Técnica
1	Long Method (cabeçalho/rodapé)	Extract Method
2	Magic Numbers/Strings	Replace Magic Number with Symbolic Constant
3	Duplicated Code	Extract Method (formatadores CSV/HTML)
4	Condicionais por papel; aninhamento	Estratégia por perfil + guard clauses
5	Comportamento com papéis desconhecidos	Dispatch com Map (18 casos comparativos também passaram)

Resultados

Função/método	Linhas antes	Cognitiva antes	Ciclomática antes	Linhas depois	Cognitiva depois	Ciclomática depois
`generateReport`	58	27	15	23	5	4
`createHeader`	-	-	-	15	2	3
`createFooter`	-	-	-	9	2	3

`formatCsvItem / formatHtmlItem`	-	-	-	3 / 4	0 / 2	1 / 3
`formatReportItem`	-	-	-	9	2	3
`formatAdminItem / formatStandardUserItem`	-	-	-	6 / 6	1 / 1	2 / 2
Total de funções	2			9		

O método principal caiu de 58 para 23 linhas e de complexidade cognitiva 27 para 5 (e ciclomática de 15 para 4). Nenhuma função do arquivo novo passa de 5 em complexidade cognitiva. As medições vêm de regras do ESLint com limite zerado só na linha de comando, para o ESLint informar o valor de cada função.

Validação final	**Resultado**
`npx eslint src/ReportGenerator.refactored.js`	0 erros e 0 avisos
`npx eslint src/`	Só os 2 problemas já conhecidos, ambos no arquivo original
`npm test`	2 suítes, 10 testes aprovados, 0 falhas (5 do original + 5 do refatorado)
Comparação extra	Original e refatorado deram saídas idênticas em 18 combinações (6 papéis x 3 formatos)

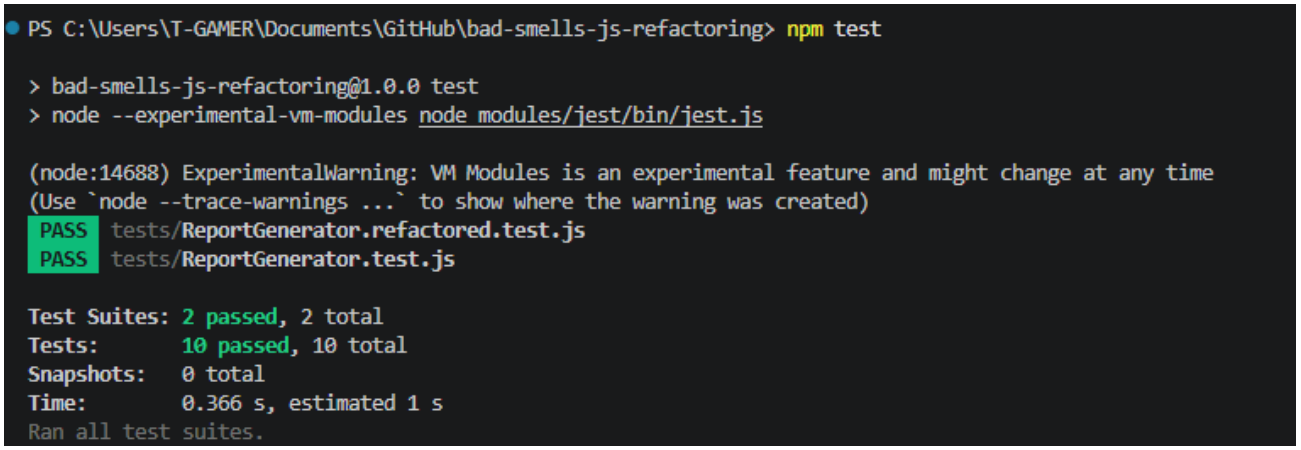


Figura 2 - Execução completa dos testes após a refatoração.

4. Conclusão

Refatorar sem testes é pisar em ovos: dá para melhorar o desenho e, sem querer, mudar o comportamento. Aqui a suíte foi o que me deixou mexer com confiança. Rodei os testes a cada passo, e ela ainda denunciou, logo no começo, um bug do código original (o espaço extra no HTML) que eu teria deixado passar. Se algum passo quebrasse, eu saberia exatamente qual foi e desfaria só ele.

Reduzir os bad smells deixou o código mais fácil de ler e de mudar: o método que concentrava tudo virou funções pequenas com nomes que explicam o que fazem, a duplicação sumiu, os limites ganharam nomes e um novo formato ou perfil agora é uma mudança localizada. Isso também facilita testar, porque cada parte é verificável isoladamente. E o ESLint com SonarJS deu números objetivos (27 contra 5) para saber se melhorei de verdade, embora não tenha pego tudo, então a leitura humana continua necessária.